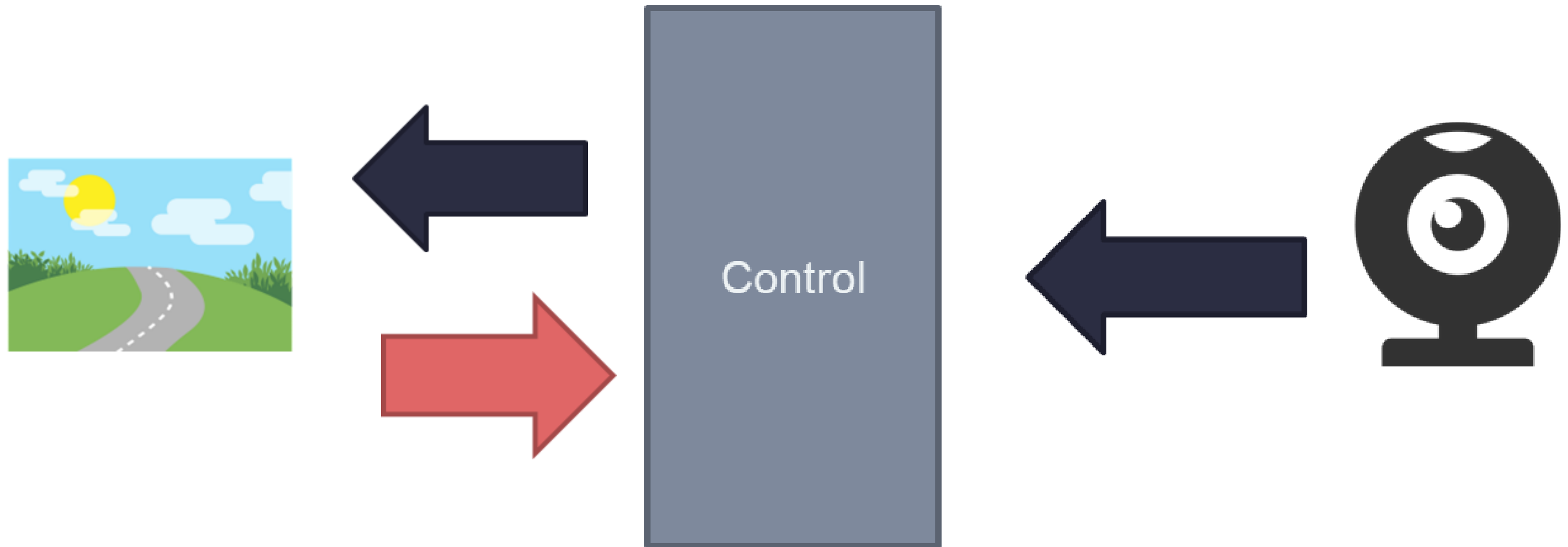


LANE KEEPING

Lane Keeping System

ทำหน้าที่ตามชื่อของฟังก์ชัน คือปรับพวงมาลัยเพื่อให้รถของอยู่ในช่องทางหรือส้น
เตีอนเมื่อรถเบี่ยงออกจากเลน เทคโนโลยีนี้สามารถช่วยให้ผู้ขับขี่ที่มีสมาธิกับสถานกา
รณ์อื่นๆ บนถนน โดยขับรถได้อย่างปลอดภัยและผ่อนคลายยิ่งขึ้น

Lane Keeping System

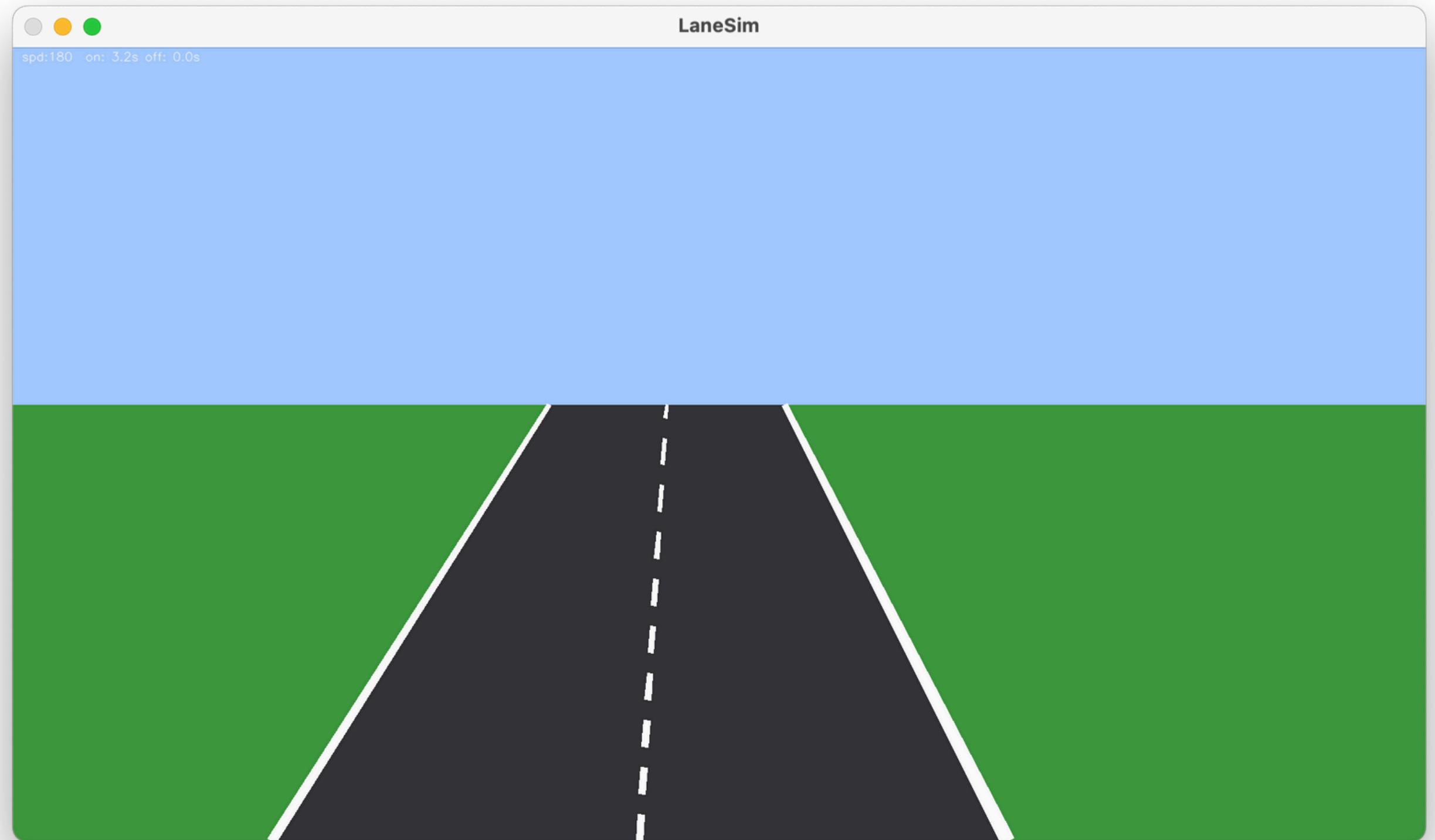


read image



```
1  import cv2
2  from lane_sim import LaneSim
3
4  sim = LaneSim(width=1920, height=1080)
5  while True:
6      key = cv2.waitKey(1) & 0xFF
7      action = -1 if key in (ord('a'), 81) else 1 if key in (ord('d'), 83) else 0
8      if key == 27: break
9      frame = sim.step(action)
10     cv2.imshow('LaneSim', frame)
11 cv2.destroyAllWindows()
```

read image

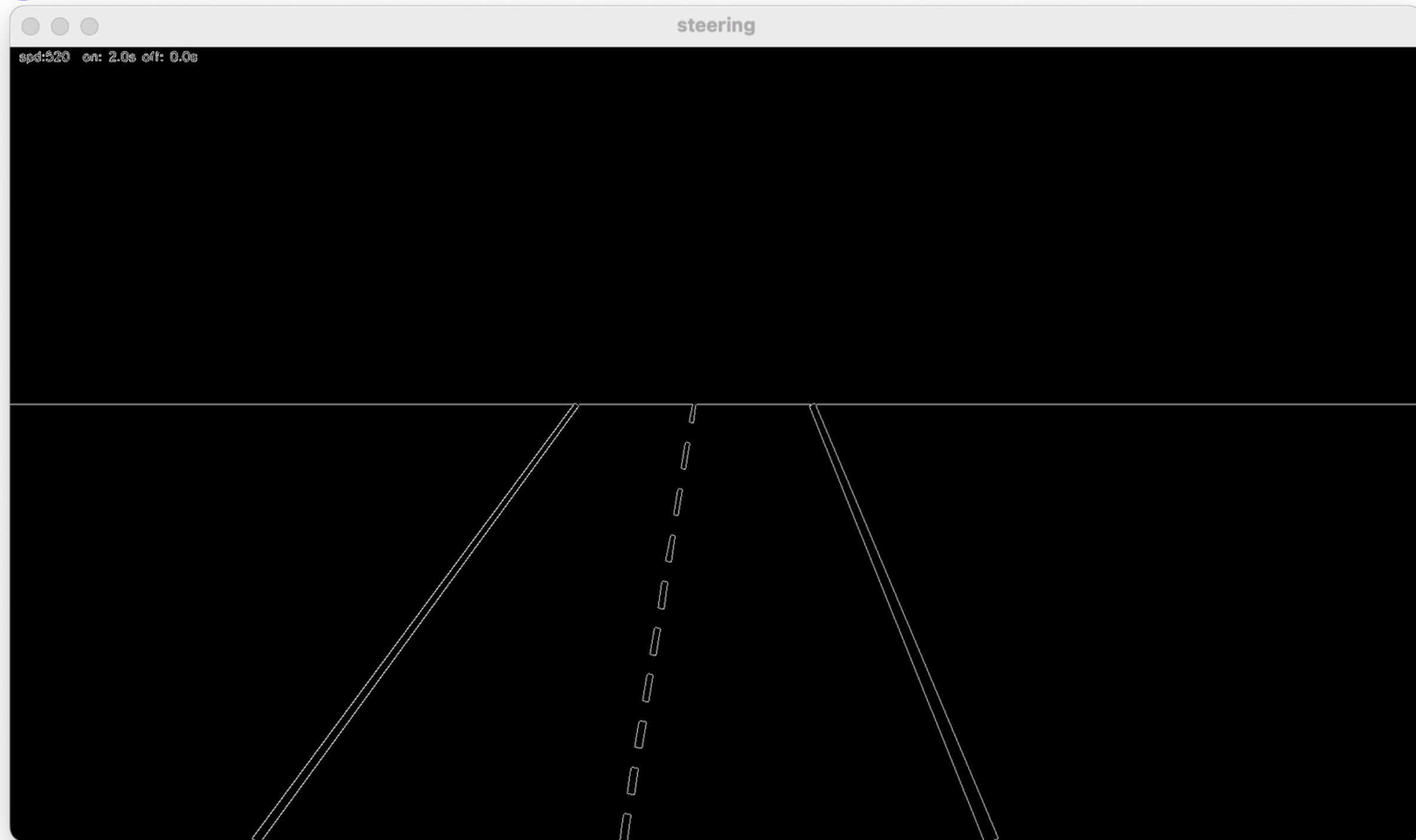


Canny



```
1 frame = sim.step(action)
2 gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
3 canny = cv2.Canny(gray, 50, 150)
```

Canny



Select Region of Interest (ROI)

การเลือกพื้นที่ที่สนใจ ที่ต้องการเน้นในเฟรม ในกรณีนี้ไม่ต้องการให้เห็นสิ่งของจำนวนมาก ต้องการให้เน้นไปที่เส้นเลน

ระบบพิกัด (แกน x และ y) เริ่มจากมุมบนซ้าย จุด (0,0) เริ่มจากมุมบนซ้าย แกน y คือความสูงและแกน x เป็นความกว้าง

`cv2.fillPoly( )` เติมสีที่สนใจลงไปในรอบให้เต็ม

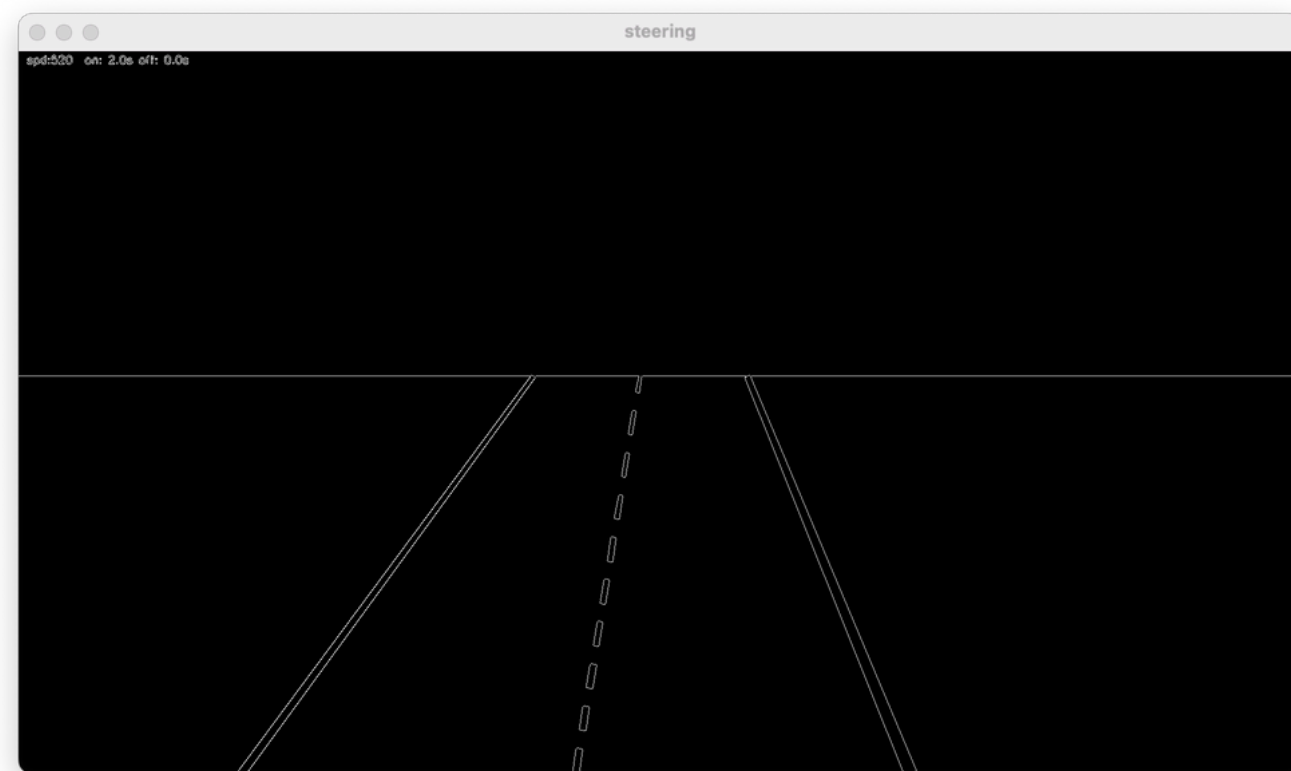
`cv2.bitwise_and(frame,frame,mask=fgmask)`

คือการนำภาพระดับบิตที่ชื่อว่า frame มาทำการ AND กับ Mask ที่ชื่อว่า fgmask โดยมีหลักการว่า 0 AND กับอะไรก็จะได้ 0

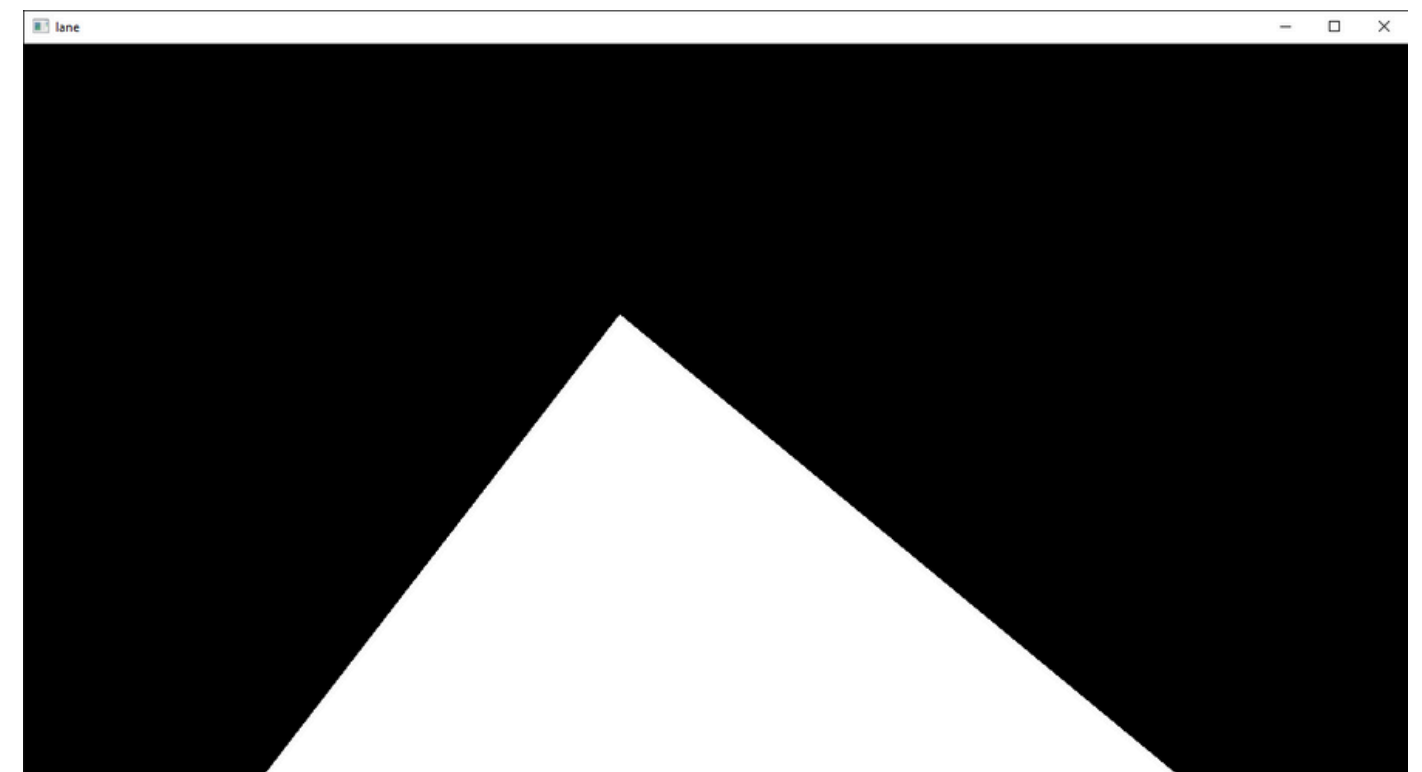
Select Region of Interest (ROI)

```
1  def average_slope_intercept(image, lines):
2      left_fit = []
3      right_fit = []
4      if lines is None:
5          return None
6      for line in lines:
7          x1, y1, x2, y2 = line.reshape(4)
8          parameters = np.polyfit((x1, x2), (y1, y2), 1)
9          slope = parameters[0]
10         intercept = parameters[1]
11         if slope < 0:
12             left_fit.append((slope, intercept))
13         else:
14             right_fit.append((slope, intercept))
15     if not left_fit and not right_fit:
16         return None
17
18     lines = []
19     if left_fit:
20         left_line = make_coordinates(image, np.average(left_fit, axis=0))
21         lines.append(left_line)
22     if right_fit:
23         right_line = make_coordinates(image, np.average(right_fit, axis=0))
24         lines.append(right_line)
25
26     return np.array(lines) if lines else None
```

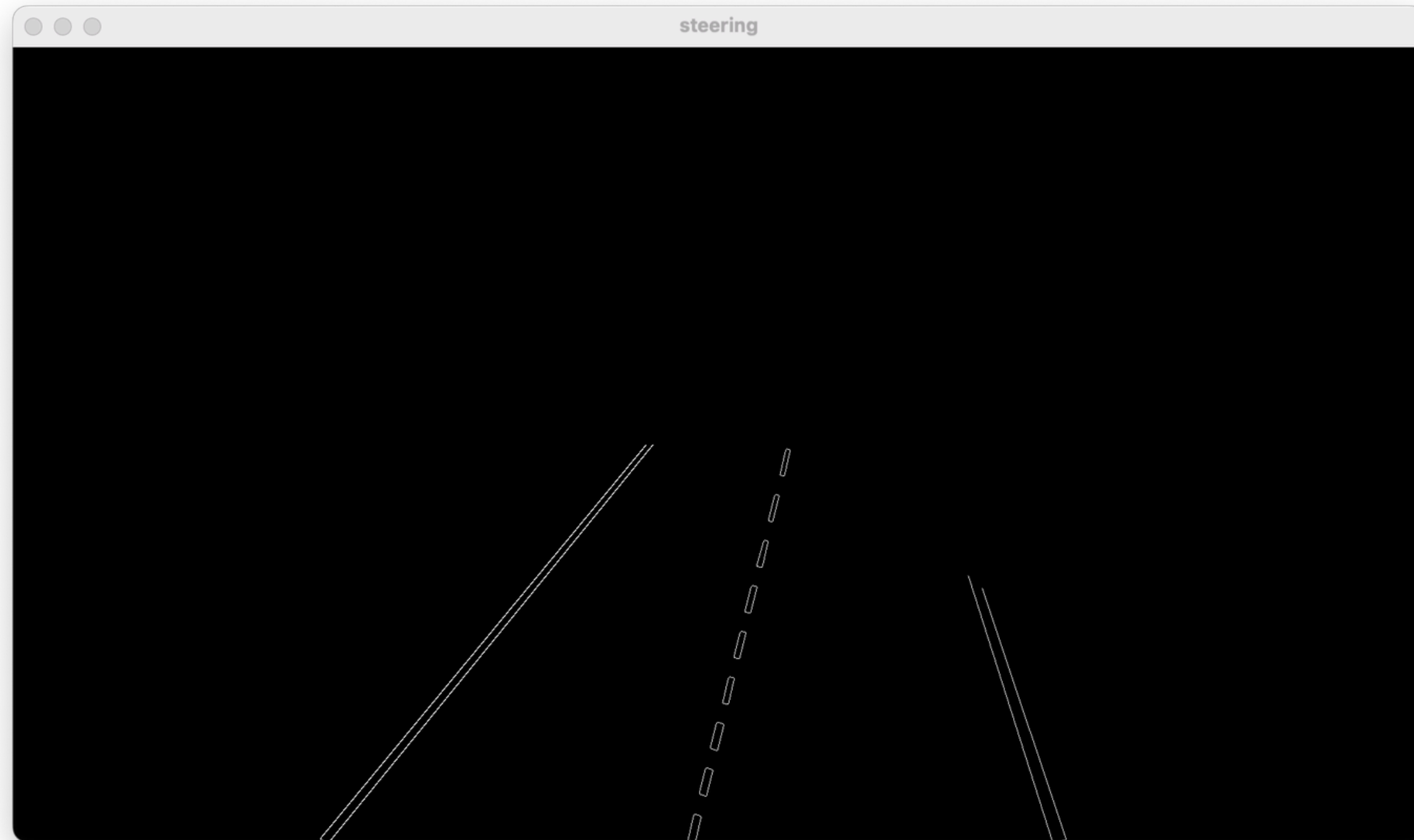
Select Region of Interest (ROI)



&



Select Region of Interest (ROI)

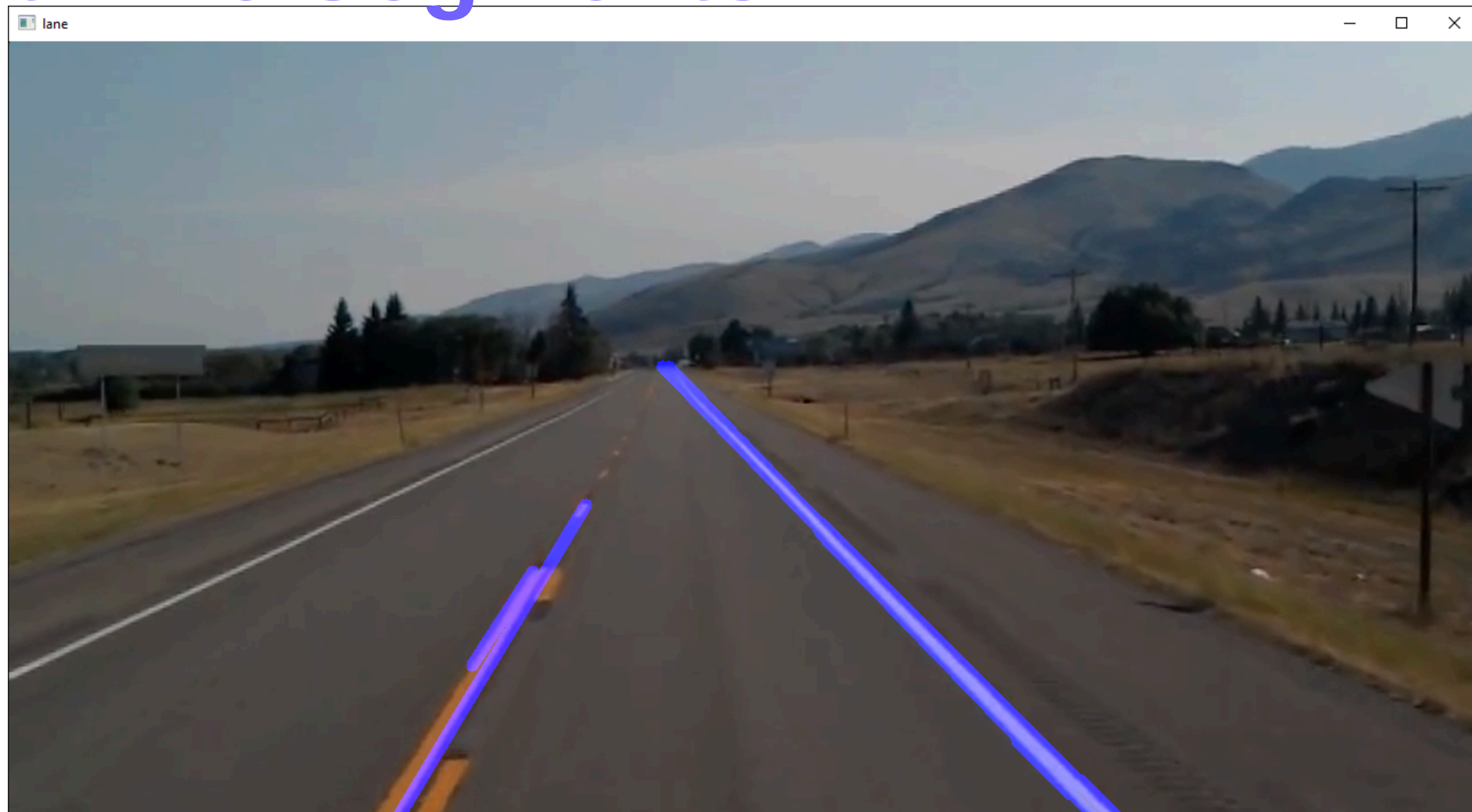


Detect Line Segments



```
1 lines = cv.HoughLinesP(roi_img,2,np.pi/180,100,np.array([]),minLineLength=40,maxLineGap=50)
```

Detect Line Segments



Average slope and Intercept

```
1 def make_coordinates(img,line_parameters):
2     slop,intercept = line_parameters
3     y1 = img.shape[0]
4     y2 = int(y1*(3/5))
5     x1 = int((y1-intercept)/slop)
6     x2 = int((y2-intercept)/slop)
7     return np.array([x1,y1,x2,y2])
8 def averaged_slop_intercept(img,lines):
9     left_fit = []
10    right_fit = []
11    for line in lines:
12        x1 , y1, x2, y2 = line.reshape(4)
13        parameters = np.polyfit((x1,x2),(y1,y2),1)
14        slope = parameters[0]
15        intercept = parameters[1]
16        if slope < 0:
17            left_fit.append((slope,intercept))
18        else:
19            right_fit.append((slope,intercept))
20    left_fit_average = np.average(left_fit,axis=0)
21    right_fit_average = np.average(right_fit,axis=0)
22    left_line = make_coordinates(img,left_fit_average)
23    right_line = make_coordinates(img,right_fit_average)
24    return np.array([left_line,right_line])
```

Average slope and Intercept

